TeamWeaver 과제 결과 보고서

SI 조직 인력 → 프로젝트 배치 최적화와 근거를 검사받는 AI 설명 (PoC) · 2026-10-18

과제 내용	기술 적합·협업 이력·월별 가용률·등급 정원·월 예산을 한 번에 계산해 배치 안을 내고, 그 근거를 검사받는 AI 글로 설명하는 의사결정 보조 시스템
수행 기간	2026-08-02 ~ 2026-10-19 (커밋 480여 개) · AI 코딩 에이전트 협업 개발: Claude 2계정(제품)과 Codex(계산 검증 단계 구현, 이후 독립 코드 리뷰)
결과물	소스 저장소(main) · 시연 영상(약 9분 30초) · 본 보고서 · 실행 `scripts/run_poc.sh`
데이터	실제 인사 자료 모양의 **가상** CSV 묶음(13종 파일, seed 고정): 100·200·300명 × 연초 계획·운영 중, 사업 20·40·60건, 사람 이름도 지어낸 이름

읽는 법 — 시연 영상은 화면 흐름을, 이 보고서는 **영상으로 확인하기 어려운 것**(측정 수치·모델 정식·검증 방법·판단 근거·한계)을 다룬다. 모든 수치는 가상 데이터와 설계한 점수표 안에서 잰 값이다. 계산이 맞다는 증거이지 **실제 사업 성과가 좋아진다는 증거가 아니다**(`NOT_CALIBRATED`).

1. 과제 개요
2. 수행 결과 요약
3. 정량적 성과
4. 정성적 성과
5. 시스템 구성
6. 주요 적용 기술
7. 주요 설계 판단과 근거
8. 사업적 가치와 확장
9. 한계와 다음 단계

부록: 재현 방법·근거 파일

1. 과제 개요

1.1 배경과 문제

SI 조직에서는 한 사람이 여러 사업에 걸쳐 일한다. 배치를 정하려면 **기술 적합**만이 아니라 사람마다 **달마다 남은 시간(가용률)**, 사업의 **등급별 정원**과 **월 예산**, **함께 일해 본 동료(협업 이력)**를 동시에 맞춰야 한다. 한 조건을 맞추면 다른 조건이 깨지기 쉬워, 담당자의 수작업 배치로는 ① 조건 위반 여부, ② 더 나은 배치가 있었는지, ③ 인사팀에 "왜 이 팀인가"를 설명할 근거를 확인하기 어렵다. 상황은 셋이다 — 가장 잦은 **신규 사업 편성**(대부분 이미 일하는 중, 쉬는·끝나 가는 사람과 몇 명만 옮겨 새 사업 팀을 짬), 진행 사업에 이슈가 생긴 **긴급 투입**, 드문 **연간 계획**(백지에서 6개월 배치).

1.2 목표와 달성

목표	달성 기준	결과
G1 모든 조건을 지키는 배치를 계산으로 낸다	100·200·300명에서 빈자리·제약 위반 없이 안 A~D, 운영 중 K 비교	**달성** — 12개 안 모두 빈자리 0·위반 0, 운영 중 12행 모두 최선 증명 (3.1·3.2)
G2 계산이 맞음을 시스템이 스스로 보인다	독립 검증을 통과한 해만 출력, 정답 대조, 최적과의 거리 표시	**달성** — 정답 대조 13/13, 모든 반환 해 독립 검증, 증명 상한과의 거리를 화면·PDF에(3.6)
G3 결정 근거를 지어낸 내용 없이 글로 낸다	AI 글이 기계 검사를 통과한 것만 나가고, 넣은 오류를 걸러 냄	**달성** — 소명 글 검사 통과 99.0%(처음 보는 데이터 100%), 주입 오류 13,316건 모두 검출(3.5)

1.3 범위와 전제

- PoC·과제 제출용이다. **실제 인사 자료는 적재하지 않았다**(적재 불가). 입력 형식은 실제 인사 자료의 항목과 규칙을 받아 설계했고, 개발·검증·시연은 같은 형식의 가상 데이터로 했다.
- 점수의 계수(협업 0.3·익숙한 사이 감점 0.2·하위 기술 인정 70% 등)는 설계값이다. 결과를 맞추려고 데이터 생성기나 계수를 조정하지 않았고, 실패·중단도 기록에 남겼다.

1.4 가상 데이터 설계

항목	설계
조직·사업	조직형 묶음 — 100명(데이터 플랫폼 실행 그룹), 200명(+AI 실행 그룹), 300명(+업무 자동화 플랫폼 실행 그룹). 사업 25·50·74건(실행·제안 + 계획 전에 시작한 장기 사업·추가 과제), 6개월 계획 기간(연간 묶음은 12개월), 등급 4단계·월 단가표·월별 가용 M/M. 운영 중 묶음은 같은 사람으로 대부분이 이미 배치된 상황(100명: 87명 배치·대기 10명·육아휴직 3명)
가용률·정원·예산·수명 주기 (v3·v4)	대부분 100%, 3% 육아휴직(해를 걸친 12개월), 2% 한 달 휴가(가용률 그대로) — 모든 묶음이 같은 휴직(1.5). 정원은 달·등급마다 일할 수 있는 인원의 85% 이하, 예산은 정원 × 단가 × 1.0~1.25배. 사업은 2024년 시작 운영(내부 2~3명)·2025년 고도화·2026 상반기 시작·운영 사업의 추가 과제(내부 2명·6개월, 외주는 참고값)가 섞인다. 사람·이력·평가 글은 그대로
이력·평가	10년 업무 이력, 기술별 경력 개월, 동료 평가(항목 점수 + 글). 평가 글 생성기 v2는 아쉬운 점을 항목마다 문장으로 쓰고, 직설형 평가자 8%, 항목 점수와 온도가 어긋난 글 5%를 섞어 "항목이 아니라 글을 읽어야" 맞힐 수 있게 했다
과거 성과(선택)	고객 평가·고객 요청 교체 기록을 모델과 독립된 숨은 규칙(실제 역량·이전 협업·업종 경험·잦은 이동·고객 궁합·운)으로 생성. 첫 버전은 평가 점수와 실제 역량의 상관이 0.97이라 "정답 베끼기"가 되므로, 평가를 "보이는 역량"과 평가자 성향으로 매기게 고쳐 0.35~0.45로 낮췄다
지킨 원칙	목표 수치를 위해 생성기·계수를 고치지 않는다. 가상 성과로 모델의 효과를 "입증"하지 않는다(자기가 만든 규칙을 다시 찾는 순환이므로)

1.5 현업 수행 경험에서 나온 요구

업무 규칙과 가상 데이터의 모양은 대부분 과제 수행자가 SI 사업을 직접 수행하며 겪은 현업 경험에서 나왔고, 결과를 직접 보며 여러 번 바로잡았다.

현업에서 본 실제(요구)	반영한 것
그룹 단위 조직, 큰 사업도 배치 대상은 내부 정원뿐. 오래전 경력은 잘 기억되지 않는다	조직형 묶음 100·200·300명, 최근 10년 이력만
평소엔 대부분 배치돼 있고 신규 제안 2~3개를 최근 끝난 10명 안팎으로 꾸린다. 진행 사업이 나빠지면 급히 더 넣는다. 손대면 안 되는 사업·사람이 있다	세 상황(신규 사업 편성·긴급 투입·연간 계획), "최대 K명만 옮길 때" 비교, 사업·사람 고정
부서장이 정하면 팀장·인사팀이 확인한다. 숫자만으로는 소명이 안 되고, 빼온 사업의 영향도 설명해야 한다	인사팀 검토 자료(잠긴 사실 + 부서장 판단·해명·이의), 결재는 사내 전자결재로(8.4)
대부분 100% 가용, 약 3%는 해를 걸친 1년 육아휴직, 약 2%는 한 달 휴가(사업이 자체 감당). 계획은 1년 단위로도 짠다	가용률 가정(생성기 v3), 쉬는 달이 걸린 사람은 사유와 함께 비활성, 계획 기간 6·12개월
사업도 나이가 섞여 있다 — 2024년에 시작해 운영 단계인 사업(내부 소수), 2025년 사업 뒤의 고도화, 2026년 상반기 시작, 운영 사업에 6달 붙는 추가 과제(회사 인력 2명, 외주 여러 명)	사업 수명 주기 섞기(생성기 v4): 원래 시작 달·단계 표시, 추가 과제는 상위 사업 연결·외주 인원 참고값(계산에 안 씀)
여러 사업을 동시에 맡는 사람은 드물고, 맡으면 두 사업 모두 100%. 사람이 부족하거나 "잠깐 봐줘야" 할 때 특정 인력이 맡는다. 예산은 정원 × 단가로 잡힌다	전일 투입 기본, 프로젝트 동시 수행은 부서장이 사람 단위로 사유와 함께 허용(빈자리 허용 후보·긴급 투입 "동시 수행으로 지원"), 예산 1.0~1.25배

2. 수행 결과 요약

+8~10%
제약을 지키는 휴리스틱(같은 시간) 대비 배치 품질, 100~300명(휴리스틱은 200·300명에서 빈자리 3·2석)

0.8%
300명 안 A와 증명 상한의 거리(빈자리 0·위반 0, 111초에 최선 증명)

0석 · +4.1%
신규 사업 편성: K=0부터 빈자리 0, 3명 옮기면 품질 +4.1%(100명), 모두 최선 증명

55%
LLM에게 100명 배치를 맡기면 솔버 품질의 55%(점수 제공·1회 수정, 10-12 데이터 기준)

99.0% / 100%
소명 글 검사 통과(조율 618편 / 처음 보는 데이터 225편, 10-12 측정), 시험 1,962개

34 → 0
나쁘게 평가한 사이가 협업 보상에 든 수: 규칙 판정 → AI 판정(100명)

2.1 구현 기능

기능	내용	영상
입력 검증	CSV 13종 묶음 업로드. 오류를 파일·줄·칸까지 표시하고, 통과하기 전에는 계산 데이터를 바꾸지 않음. 앱 전체 로그인, 편성 기록(JSON) 내보내기·불러오기	D1
연간 계획	안 A~D(대안은 구성이 다르고 품질 ≥ A의 95%), 계산 신뢰도 배지, 안 A부터 먼저 보여 주는 점진 반환, 계산 멈추기, 지난 편성 다시 열기	D2
인사팀 소명 글	"왜 이 팀인가"를 지식 그래프 사실로 작성, 근거 문구 ↔ 사실 연결, 불리한 사실 표시·덧붙임	D3
교체 검토	한 사람 교체의 점수 변화를 전체 목적·제약으로 재채점(참고값) + 동료 평가 인용 AI 설명, 위반이면 결론 "보류"	D4
신규 사업 편성	신규 사업 등록(정해진 양식·기술 사전·이전 사업 복사), 후보 세 갈래(쉬는·끝나 가는·진행 사업에서 빼 오기, 이전 팀원 표시), 최대 K명(0~3) 이동 자동 편성, 사업·사람 고정, 확정·되돌리기(작업 데이터의 판)	D5
긴급 투입	밤사이 뽑아 둔 대기 명단, 최선 n명 조합(빼 온 사업의 빈자리 메움까지), "프로젝트 동시 수행으로 지원"(원래 사업 유지, 부서장이 사유와 함께 허용 — 빈자리가 남으면 허용 후보도 제안), 확정 때 정원·예산·요구 기술 증원	D6
인사팀 검토 자료	확정마다 사업별 소명 자료: 잠긴 시스템 사실(옮겨 온 사람의 원래 사업 영향·메움 포함) + 부서장이 고친 본문 + 판단·원래 사업 합의 + 사실마다 해명·이의 → PDF	D7
PDF·기술 지도·설정	서명 확인 뒤 서버가 다시 계산한 PDF, 조직 기술 지도(공급/수요), 관리자 설정(투입 방식 — 전일 투입 기본, 익숙한 쌍 기준·계산 시간·동시 탐색 수)	D8

규모: 제품 코드 약 1.9만 줄(코어 8.2천·API 6.0천·웹 5.2천), 시험 코드 약 2.4만 줄(파이썬 2.1만·웹 3.3천), 실험·리허설 도구 1.6만 줄, 측정 보고서 `rehearsal/results/` 17개.

2.2 수행 경과

기간	단계	핵심 결과
08-02 ~ 08-21	기반 구축	점수·MILP 코어, 저장 기술 실험(그래프 DB Neo4j 우위 0/20 → 제거), API·실시간 반환(SSE)·교체 검토·웹·PDF
09-18 ~ 10-03	계산 검증	원시 해 계약·독립 검증기·정답 대조 11/11(Phase 0), 오픈소스 솔버 3종 357칸 비교(Phase 1)
10-04 ~ 10-05	제품 연결	HiGHS 고정·독립 검증 강제·대안 필터, 실제 인사 자료 모양의 CSV 입력 계약, 300명 리허설로 보상 쌍 200 결정
10-06 ~ 10-07	규모·운영	익숙한 쌍 규칙, 시드 4 동시 탐색, 평가 글 LLM 판정, 운영 중 편성(K 비교·보강), 시연 묶음·미리 계산
10-07 ~ 10-08	AI 비교 측정	평가 글 판정(E4)·교체 설명(E5)·LLM 직접 배치(E6), 외부 AI 대 사내 가정 모델
10-08 ~ 10-11	지식 그래프·소명	기술 이름 사전·하위 기술 인정, 지식 그래프, GraphRAG 소명 글(E7b → E7c), 조직 기술 지도, 사내 LLM 교체 측정
10-12 ~ 10-14	감사·마무리	전체 감사 27건 수정, 계산 멈추기·PDF 계산 신뢰도, 깨끗한 복제본 재현, 시연 최종 리허설, 제출 문서
10-18	평가 반영·서비스화	제3자 관점 평가(Codex)를 받아 비교 상대를 강화하고 AI 판정의 배치 영향을 측정. 앱 전체 로그인, 편성 기록(JSON) 내보내기·불러오기, 화면 전면 개편
10-19	3가지 상황 · 현업 규칙 재설계	신규 사업 편성(첫 화면)·긴급 투입·연간 계획(1년)으로 재편, 작업 데이터의 판, 확정마다 인사팀 검토 자료. 가용률(휴직·휴가)·전일 투입·사람 단위 동시 수행(1.5)으로 모델·데이터를 다시 만들고 모든 숫자를 다시 쟀다

3. 정량적 성과

3.1 연초 계획: 배치 품질과 계산 성능

규모	사업	안마다 한도	안 A 목적값	상한 대비 차이(A~D)	빈자리·위반	안 A~D 합계	휴리스틱 → 최적화(배치 품질)
100명	25	60초	70.78	2.3~4.9%	0 · 0	2.4분	65.46 → 70.78 (**+8.1%**)
200명	50	90초	163.71	1.2~4.6%	0 · 0	4.8분	148.79 → 163.71 (**+10.0%**, 휴리스틱 빈자리 3석)
300명	74	120초	246.40	0.8~2.4%	0 · 0	5.7분	224.34 → 246.40 (**+9.8%**, 휴리스틱 빈자리 2석)
100명 · 12개월	44	240초	132.52	3.4~5.0%	0 · 0	9.0분	—

- 전일 투입(1.5·6.1), HiGHS 시드 4개 동시 탐색, 탐욕 초기해. "상한 대비 차이" = (증명 상한 − 목적값) ÷ 목적값. 안 A는 300명에서 0.8%로 최선 증명(111초), 100·200명·연간에서는 1% 차이까지 증명하려다 시간 한도에서 멈췄다. B~D는 허용 차이 안 최선 증명. 사업 수에는 계획 전에 시작한 장기 사업·추가 과제가 들어 있다(생성기 v4). 배치 품질 = 기술 + 협업 − 익숙한 쌍(빈자리 감점 제외). 출처 `demo/precomputed/`, 시간 한도 근거 `rehearsal/results/time-budget-full.json`(허용 차이 5% 증명까지 100·200·300명 26·55·66초 × 여유 1.5 → 권장 60·90·120초, 12개월은 ×4).
- 휴리스틱 = 예산·가용률을 지키며 기술 적합과 팀원 협업을 함께 보고 자리를 채운 뒤 **같은 시간** 동안 같은 등급 사람 바꾸기를 반복하는 강한 비교 상대(`rehearsal/strong_baseline.py`, 같은 평가기). 전일 투입에서는 배정 문제에 가까워져 예전 투입률 나누기 방식(+24~31%)보다 차이가 작지만, 휴리스틱은 200·300명에서 자리를 다 채우지 못했다. 예전 방식은 시간을 늘릴수록 좋은 사람을 30%씩 쪼개 넣었다(300명 300초: 81명이 여러 사업, `rehearsal/results/concurrency-compare.json`) — 전일 투입으로 바꾼 근거다.

3.2 신규 사업 편성: 최대 K명만 옮길 때

규모(상황)	배치 품질 K=0 → 1 → 2 → 3	개선(K=3)	빈자리	최선 증명	K별 최대 시간
100명(87명 배치·대기 10·육아휴직 3·신규 제안 2 + 추가 과제 1)	66.58 → 67.74 → 68.71 → 69.31	+4.1%	0	4/4	2.4초
200명(175명 배치·대기 20·육아휴직 5·신규 제안 3 + 추가 과제 1)	119.26 → 120.63 → 121.62 → 122.53	+2.7%	0	4/4	15.1초
300명(262명 배치·대기 30·육아휴직 8·신규 제안 3 + 추가 과제 1)	177.02 → 178.40 → 179.54 → 180.53	+2.0%	0	4/4	31.9초

갭 0으로 풀어 모든 행이 최선 증명이다. 신규 제안과 운영 사업에 붙은 추가 과제(내부 2명)의 정원은 대기 인력의 등급 구성에 맞춰 만들어(생성기 v3·v4) 아무도 옮기지 않아도(K=0) 다 채워지고, 옮길수록 기술이 더 맞는 사람을 데려온다. 운영 중 묶음은 전원이 이미 100%로 배치된 상태라 전일 투입과 그대로 맞는다. 출처 `demo/precomputed/org-n*-operating.json`.

E5·E6·E7c(3.3·3.5·3.6)는 10-12 데이터(예전 투입 방식) 기준이다 — 외부 AI 비용 때문에 다시 재지 않았다. 3.4의 판정 정확도(E4)는 평가 글이 그대로라 지금도 맞고, 배치 영향은 10-19에 다시 쟀다.

3.3 LLM에게 배치를 직접 맡기면 (E6)

문제	조건	쓸 수 있는 배치	품질(솔버 대비)	응답 시간(솔버)
20명·사업 4	원자료만 / 우리 점수 제공 + 1회 수정	2/2 · 2/2	89.0% · 94.2%	222초 · 236초 (0.3초)
50명·사업 10	〃	0/2 · 2/2	— · 69.6%	423초 · 291초 (0.9초)
100명·사업 20	〃	0/2 · 2/3	— · 55.3%	661초 · 347초 (30.6초)

외부 LLM(gpt-6-luna, 추론 high)에게 같은 규칙·목적을 설명하고 JSON 배치를 받아 서비스 평가기로 채점했다. "쓸 수 있는 배치" = 제약 위반 0이고 빈자리가 솔버보다 많지 않은 답. 원자료만 주면 50·100명에서 모두 위반·빈자리가 났다. 칸마다 2~3회라 표본이 작고, 100명 솔버 해는 시간 한도 해라 % 비교는 보수적이다. → **배치 결정은 솔버가, LLM은 글을 맡는 구조**의 근거(`rehearsal/results/llm-vs-solver.html`).

3.4 평가 글은 AI가 읽어야 한다 — 판정 방식 비교(E4)와 배치에 미치는 영향

동료 평가는 체크한 항목(좋은 점·아쉬운 점)과 서술 글로 이뤄진다. 항목만 세는 **규칙**, 감성 점수 API **Jev**, LLM 두 가지(서비스용 **OpenAI gpt-6-luna**, 사내 후보 **GLM-5.3-Flash**)를 같은 글에 돌렸다.

판정 방식	① 글로만 드러난 부정 (직설·혹평 34건)	② 돌려 말한 부정 (정답 있는 글 300건 중 부정 83건)	1,000건 비용	판단
규칙(체크한 항목만 셈)	4/34 (12%)	— (항목이 곧 정답인 시험)	0	글을 읽지 않아 글에 쓴 부정을 놓친다
Jev(감성 점수 API)	32/34 (94%)	14/83 (17%)	$0.02	노골적인 부정만 읽는다
GLM-5.3-Flash(사내 후보)	34/34 (100%)	43/83 (52%)	$0.09	완곡한 지적의 절반을 놓친다
OpenAI gpt-6-luna(서비스)	**34/34 (100%)**	**74/83 (89%)**	$0.08	**채택** — 정답과 상관 0.94

① 시연 원문 무작위 300건 중 생성기가 "직설형 평가자"·"항목보다 독한 글"로 만든 34건. 반대로 항목은 아쉬운 점이 많은데 글은 부드러운 4건은 규칙만 부정으로 읽었다. ② 고른 항목대로 충실하게 쓴 글(정답 = 항목 균형) — 글만 보고 숨은 부정을 찾아내는 독해력 시험이다. 판정 기준은 측정 전에 커밋했다.

판정 차이는 배치 결정까지 바꾼다. 같은 시연 묶음을 규칙 판정과 AI 판정으로 각각 만들어 안 A를 서비스와 같은 설정(시드 4개·안 A 허용 차이 1%)으로 풀었다(`rehearsal/ai_effect.py`).

규모	글로만 드러난 부정 평가	그런 사이가 협업 보상 대상에 든 수 규칙 → AI	그런 사이를 같은 팀에 규칙 → AI	팀 협업 점수 규칙 → AI
100명	145건 / 1,372	34 → **0**	9 → **1**	5.18 → 7.79 (+50%)
200명	255건 / 2,641	34 → **0**	14 → **5**	12.15 → 15.26 (+26%)
300명	346건 / 4,012	22 → **0**	12 → **7**	14.22 → 18.94 (+33%)

규칙으로 판정하면 서로 나쁘게 쓴 사이가 "궁합 좋은 상위 200쌍"에 들어가 함께 배치될수록 점수를 받는다. AI 판정은 세 규모 모두 그런 사이를 보상에서 뺐다. 팀 협업 점수는 AI가 읽은 평가로 다시 채점한 값이고, 두 안 모두 제약 위반 0이다(2026-10-19 전일 투입·생성기 v4로 다시 잼, `rehearsal/results/ai-effect.json`).

3.5 그 밖의 생성형 AI 구성요소 (서비스 운영 모델: 외부 gpt-6-luna)

쓰임	측정	결과	비용·시간
교체 설명(E5)	운영 중 교체 100건	바로 사용 **93%**, 새 위반 교체의 "보류" 준수 90% — 어긴 글은 서버가 버리고 규칙 설명으로 대체. 지시문 강화 뒤 재측정: 보류 18/20, 인정 경력 출처 표기 9/10(빠지면 서버가 덧붙임)	평균 3.5초, 100건 $0.06
인사팀 소명 글(E7c)	사업 × 3회 843편(조율 데이터 618 + 처음 보는 데이터 225), 사실을 일부러 틀리게 한 오류 주입	검사 통과 **99.0%**(612/618), 처음 보는 데이터 **100%**(225/225). 주입 오류 **13,316/13,316** 검출, 채택 글 무결성 837/837	편당 약 8초·$0.0009
기술 이름 사전	기술 이름 절반을 다른 표기로 바꾼 사본	사전을 쓰면 기술 점수 차이 0. 사전이 없으면 100명 쌍의 86% (1,714/2,000)에서 점수가 바뀌고, 그렇게 짠 배치를 바르게 재면 57.8 → 49.1	—

판정 기준은 측정 전에 커밋했다(사전 등록). 출처: `outputs/review-judge-comparison.html`, `rehearsal/results/ai-effect.json`, `outputs/briefing-llm-comparison.html`, `rehearsal/results/justify-scale-e7c.html`, `rehearsal/results/skill-dictionary.html`.

3.6 사내 LLM으로 바꾸면 (사내 서버를 쓸 수 없어 GLM-5.3-Flash로 가정)

쓰임	외부 gpt-6-luna	사내 가정 GLM-5.3-Flash	뜻
평가 글 부정 검출	89%	52%	노골적인 부정은 똑같이 잡지만 돌려 말한 지적의 절반을 놓친다 — 사내 전환 때 더 큰 모델이나 지시문 보강이 필요
교체 설명	바로 사용 93% · 보류 90% · 3.5초	97% · 100% · 10.3초	규칙은 더 잘 지키고 느리다
소명 글(처음 보는 75건)	채택 100% · 8초	97.3% · 중앙값 60초	정확도는 비슷(차이 유의하지 않음), 대기 상한을 40초 → 300초로 늘려야 한다

서비스는 외부 AI로 운영하고 사내 전환 시 차이를 여기 남긴다(사용자 결정 2026-10-11). 실데이터는 외부 AI로 보내지 않는다 — 사내 주소이거나 관리자가 명시 허용했을 때만 AI를 부른다.

3.7 정확성 검증

층	방법	결과
자동 시험	`uv run --group benchmark pytest -q`, 느린 전 과정 E2E, 웹 단위 시험·타입 검사	1,962 passed · 1 skipped / E2E 20 passed / 웹 242 passed(10-19)
정답 대조 (Phase 0)	작은 사례를 사람 조합 완전 열거 + LP로 푼 정답과 비교 (제품 MILP 코드를 읽지 않는 별도 구현), 예산·가용률을 늘리면 점수가 줄지 않는지 등	13/13 통과(정답 대조 9/9 차이 0 — 전일 투입·동시 수행 허용 사례 2개 포함, 단조성 2/2 …). 모델을 바꿀 때마다 다시 돌림
솔버 비교 (Phase 1)	CBC·HiGHS·SCIP에 같은 동결 입력, 1스레드·케이스당 240초. 본평가 288칸(4규모 × 4조건 × 3시드 × 3솔버 × 2반복) 포함 357칸	정답 사례 21건은 세 솔버 모두 통과. 나머지 솔버당 112건 중 시간 안 완료 HiGHS 79 · SCIP 75 · CBC 22(HiGHS·SCIP는 완료 건 모두 품질 확인, CBC는 상한을 내지 않아 확인 불가) → HiGHS 채택
실행 중 독립 검증	솔버의 모든 반환 해를 원자료로 다시 계산(6.4)	통과한 해만 화면·PDF로. 불러온 편성 기록도 지금 규칙으로 다시 채점
리허설·재현	실제 서버로 100/200/300명 전 과정, 시연 대본 최종 리허설, GitHub에서 새로 받은 복제본으로 설치·빌드·전체 시험	대본 숫자와 화면 일치. 시간에 따라 갈리던 시험 1개를 찾아 고침
독립 코드 리뷰	변경마다 Codex 리뷰, 지적은 고치기 전 코드에서 실패하는 시험으로 고정	전체 감사 27건 수정(10-12), 최종 점검 2건 수정, 전일 투입 재설계 리뷰 4라운드(Codex 2 · Codex 한도 소진 중 Claude Opus 대체 2)

4. 정성적 성과

- **계산 결과를 얼마나 믿어도 되는지 함께 보여 준다.** 모든 결과에 "독립 검증 통과", "최선 증명" 또는 "최적값이 이 해보다 최대 몇 % 높을 수 있음", "시간 한도 도달·사용자가 멈춤·기록에서 불러옴"을 붙이고, 교체 점수는 "참고값 · 재최적화 아님"으로 밝힌다. PDF의 이 문구도 서버 서명이 맞을 때만 찍힌다.
- **생성형 AI를 "검사받는 부품"으로 썼다.** 결정은 수학 최적화가, 읽기와 쓰기는 AI가 맡는다는 경계를 실험으로 정했다 — LLM 직접 배치(3.3)는 규모가 커지면 무너지고, 평가 글 판정(3.4)은 규칙으로는 안 된다. AI의 글은 기계 검사를 통과해야 나간다: 소명 글은 AI가 사실 번호와 연결 말만 고르고 숫자·사실 문구는 서버가 채우며, 교체 설명은 인용을 원문과 한 글자씩 대조하고 출처·보류 결론을 확인한다. 검사에 걸리면 규칙 기반 글로 대신한다.
- **규칙을 측정으로 정했다.** 보상 쌍 수·익숙한 쌍 기준·감점 방식·솔버·소명 글 방식을 대안과 같은 데이터로 비교하고, 판정 기준을 측정 전에 커밋했다(7장).
- **측정한 대로 보고한다.** 사업 효과는 `NOT_CALIBRATED`로 표시하고, 가상 성과로 효과를 "입증"하는 것은 순환 검증이라 하지 않았다. 대안 C가 A보다 조금 높은 것처럼 직관에 어긋나는 결과도 그대로 둔다.
- **실제 도입을 염두에 둔 설계.** 실제 인사 자료와 같은 입력 계약·검증 리포트, 평가 원문을 화면·AI에 보내지 않고 항목 라벨만 쓰는 비공개 모드, 외부 AI 차단 규칙, 사내 LLM 전환 시 차이 측정(3.5).
- **AI로 개발 과정 자체를 운영했다.** Claude 두 계정과 Codex가 파일 소유 영역을 나눠 병행 개발하고(작업 분담표·인수인계 기록), 코드 변경은 커밋 전에 독립 리뷰(주로 Codex)를 받았다. 리뷰 지적은 고치기 전 코드에서 실패하는 시험으로 먼저 고정하고, 측정은 판정 기준을 먼저 커밋한 뒤 돌렸다 — 11주 동안 커밋 480여 개, 시험 1,962개.
- **쓰는 사람의 시간을 아낀다.** 안 A부터 먼저 보여 주고, 오래 걸리면 멈춰 그때까지의 가장 좋은 배치로 끝낼 수 있다. 운영 중에는 "몇 명만 옮기면 얼마나 좋아지나"를 표 하나로, 인사팀 소명 글은 근거와 함께 자동으로 낸다.

5. 시스템 구성

5.1 구성도

입력

- **CSV 묶음** 사람·단가표·기술 경력·업무 이력·월별 가용·사업·등급/기술 요구·동료 평가·현재 배치 등 13종
  `core/ingest`
- **엄격 검증·변환** 오류 파일·줄·칸, 경력 개월 → 대리 레벨, 기술 이름 사전 정규화·하위 기술 인정
  `core/ingest/loader·skills`

지식

- **관계 그래프** 누가 누구와 몇 달 일했나, 익숙한 쌍
  `core/graph/memory_graph`
- **근거 색인** 동료 평가 원문·인정 경력 (SQLite)
  `core/graph/sqlite_store`
- **지식 그래프** 사업·기술·사람 사실, 기술 지도
  `core/kg`

계산	**점수** 기술 적합 S · 협업 C `core/scoring`	**MILP** HiGHS 시드 포트폴리오 · 대안 · 운영 중 K `core/optimize`	**독립 검증 · 평가기** 원자료로 재계산, 모든 배치를 같은 기준으로 채점 `validation · core/evaluate`
AI	**평가 글 판정** 극성 → C `api/review_judge`	**교체 설명(RAG)** + 인용 원문 대조 `api/rag/briefing`	**소명 글(GraphRAG)** + 사실·문법 검사 `core/kg/justify`
서비스·화면	**FastAPI** 최적화(SSE)·교체·운영 중·보강·소명·데이터·설정, 결과 서명 `api/`	**웹** React · TypeScript `web/`	**PDF** 서명 확인 → 서버 재계산 → Chromium 출력 `api/pdf`

흐름: 입력이 검증을 통과하면 활성 데이터가 바뀌고, 점수·그래프를 만든 뒤 MILP를 푼다. 솔버의 해는 독립 검증을 통과해야 API가 내보내고, AI 글은 각 검사기를 통과해야 내보낸다(실패 시 규칙 기반 글).

5.2 기술 스택

영역	사용 기술	영역	사용 기술
언어	Python 3.12 · TypeScript 6	API	FastAPI, 서버 전송 이벤트(SSE)
최적화	PuLP + HiGHS(highspy), 비교용 CBC·SCIP	AI	OpenAI 호환 API — 외부 gpt-6-luna, 사내 가정 GLM-5.3-Flash
데이터 계약	Pydantic 2, CSV 묶음(zip) + manifest	웹	React 19 · Vite 8 · Tailwind 3
저장	인메모리 그래프, SQLite 근거 색인, 서버 데이터 폴더(업로드·교체 이력·서명 키)	보고서·시험	Playwright(Chromium) PDF · pytest · Vitest

5.3 신뢰 경계

- **결과 서명**: 계산 결과마다 서버 서명(`plan_token`)과 계산 신뢰도 서명을 붙인다. 교체 적용·소명 글·PDF는 서명이 맞는 결과만 받고, PDF는 명단으로 지표를 서버가 다시 계산해 찍는다 — 화면에서 바꾼 숫자는 들어가지 않는다.
- **AI 경로**: 실데이터 모드에서는 사내 주소이거나 관리자가 명시 허용했을 때만 AI를 부르고, 평가 원문은 화면·AI에 보내지 않는다.
- **접근**: 앱 전체 로그인 — 로그인하지 않으면 모든 API가 401이다(HttpOnly·SameSite=Strict 세션, 5회 실패 시 1분 잠금, 로그아웃 시 세션 일괄 무효). 시연 기본 계정은 로컬 주소에서만 뜨고, 외부 주소는 비밀번호를 정해야 서버가 시작한다.
- **자원 보호**: 새 계산은 동시 2개(초과 시 429), 시간 한도 상한 900초, 요청 본문 2 MiB. 화면 연결이 끊기면 시드 프로세스까지 즉시 정리한다. PDF 브라우저는 서버 자기 주소만 열고 외부 요청을 막는다.

6. 주요 적용 기술

6.1 배치 최적화 모델 (혼합정수계획, MILP)

목적(최대화)	$\Sigma_{i,j}\, S_{ij}\cdot z_{ij} + \lambda\cdot\Sigma_{(p,q)\in P,\, j}\, C_{pq}\cdot y_{pqj} - \mu\cdot\Sigma_{(p,q)\in F,\, j}\, y_{pqj} - 100\cdot\Sigma_{j,g}\, \mathrm{slack}_{jg}$ $(\lambda = 0.3,\ \mu = 0.2)$
전일 투입 자격	$z_{ij} = 0$ (사업 j 기간 중 가용률$_{im}$ < 1인 달이 있으면 — 육아휴직 등)
프로젝트 동시 수행	$\Sigma_{j:\, m\in j}\, z_{ij} \le$ 동시 수행 상한$_i$ (달 m마다. 기본 1, 부서장이 사유와 함께 허용한 사람만 2)
월 예산	Σ_i 단가$_i\cdot z_{ij} \le$ 예산$_j$ (동시 수행자는 사업마다 전액)
등급 정원	$\Sigma_{i\in g}\, z_{ij} + \mathrm{slack}_{jg} =$ 요구 인원$_{jg}$ (사업에 적힌 등급 g만)
쌍 선형화	$y_{pqj} \le z_{pj},\ y_{pqj} \le z_{qj},\ y_{pqj} \ge z_{pj} + z_{qj} - 1$

- 변수: `z` 배치 여부(0/1) — **전일 투입**(서비스 기본)이라 투입률 변수가 없다. `y` 두 사람이 같은 사업에 있는지(곱의 선형화), `slack` 등급 정원 미충원(큰 감점, 결과에 노출). 예전 투입률 나누기 방식(0.3~1.0, 최대 3개 사업)은 설정에 남겼다. 시간 한도 전에 탐욕 해를 초기해로 넣어 결과가 그보다 나빠지지 않는다.
- **S**(기술 적합) = 요구 기술마다 min(보유 레벨 ÷ 요구 레벨, 1)을 중요도(1~5)로 가중 평균. 원천에 숙련 레벨이 없어 경력 개월을 대리 레벨로 바꾼다(경계 12·36·60·96개월). **C**(협업) = 0.4·min(함께한 개월 ÷ 12, 1) + 0.6·동료 평가 점수(평가 글 극성은 LLM 판정).
- **P** = |C| 상위 200쌍만 보상한다 — 쌍 변수는 (쌍 수 × 사업 수)라 5,000쌍이면 200명에서 600초에도 빈자리 57석, 300명은 해가 없었다. **F** = 익숙한 쌍(최근 36개월 중 12개월 이상 같은 사업) 전부를 감점 — 오래 붙어 다닌 사이만 몰리는 팀을 흩는다.

- 필수 기술은 하드 제약이 아니라 S로 유도하고(실제 자료 답변 "선호"), 정원에 적히지 않은 등급은 예산·가용률 안에서 자유롭게 고를 수 있다.

6.2 대안과 운영 중 증분 최적화

- **대안 B~D**: 앞선 안과 (사람, 사업) 겹침 ≤ 80%인 다양성 컷 + 품질 하한(A − 5%). 빈 팀·중복·A보다 빈자리가 많은 후보는 버리고, 없으면 "조건을 만족하는 대안 없음"으로 억지로 채우지 않는다.
- **운영 중 K**: 기존 배치를 그대로 두고 사람마다 0/1 이동 변수를 두어 Σ ≤ K. 옮긴 사람은 "원래 없던 사업"에 들어가야 한다(빼서 놀리는 해 차단), 잠근 사람은 옮기지 않는다. 빈자리 감점(자리당 100)이 섞인 목적에서 5% 허용 차이는 1~2명 이동의 개선(1~2점)을 통째로 가리므로 **허용 차이 0**으로 풀어 최선을 증명한다.

6.3 솔버 운영

- **시드 포트폴리오**: 같은 모델을 시드만 바꾼 HiGHS 프로세스 4개로 동시에 풀어 가장 좋은 해를 고른다. 1시드면 조밀한 데이터에서 대안이 품질 하한에 걸려 안 A만 나오는 일이 있었다. 모델은 프로세스 시작 뒤 전용 파이프로 넘긴다 — 시작 인자로 넘기면 자식이 시작 중 죽을 때 부모가 멈췄다(실측).
- **증명 상한**을 함께 기록해 "최적값이 최대 몇 % 높을 수 있음"으로 보여 준다. 전일 투입 권장 한도 100명 60초·200명 90초·300명 120초, 12개월은 ×4(측정 기반, 관리자 설정).
- **계산 멈추기**: 사용자가 멈추면 풀던 안은 그때까지의 해로 끝나고(종료 사유 "사용자가 멈춤", 시간 한도와 구분) 다음 안은 시작하지 않는다. 연결이 끊기면 즉시 중단한다.
- **편성 기록**: 한 번의 편성(원천 CSV · 평가 글 판정값 · 계산 설정 · 편성안 · 운영 중 K 비교)을 JSON 파일 하나로 내보내고 다시 불러온다. 불러올 때 원천을 업로드와 같은 검증으로 다시 만들고 편성안을 지금 평가기로 다시 채점해 위반이 있으면 받지 않는다. 기록이 주장하는 솔버 증명은 이 서버가 확인할 수 없어 버린다 — 화면·PDF는 "재채점 통과 · 기록에서 불러옴"으로 표시하고, 증명은 다시 계산해야 붙는다.

6.4 독립 검증기와 평가기

- 솔버가 낸 원시 해를 제품 MILP 코드와 별개로 다시 계산한다: 결측·NaN·범위·0/1, 월별 가용률·월 예산·등급 정원·동시 사업, 쌍 범위, 목적 4항, 그리고 화면에 내보낼 명단. 검증할 수 없는 값을 0으로 채우지 않는다.
- HiGHS는 z = 1.0000000000000007 같은 값을 돌려준다 — 1e-9 안쪽의 경계 잔차만 경계로 옮기고(원본 보존), 예산의 극미세 초과(상대 1e-7 이하)는 투입률 보정 LP 뒤 다시 검증한다. 이 처리 없이는 HiGHS 해가 전부 "검증 실패"였다(실측).
- 평가기는 어떤 배치든 현행 목적·제약 전체로 다시 채점한다 — 교체 검토(참고값), 보강, 단순 규칙 비교, 편성 기록 확인, PDF가 모두 같은 잣대를 쓴다.

6.5 검사받는 생성형 AI

쓰임	AI가 하는 일	기계 검사	실패하면
평가 글 판정	동료 평가 글의 극성(긍정/부정)을 판정 → 협업 점수 C에 반영. 판정값은 데이터 버전에 포함	출력 형식·범위	평가 항목 점수 사용
교체 설명 (RAG)	근거 색인에서 뽑은 동료 평가·점수 변화로 설명, 평가를 출처 번호와 함께 인용	인용이 원문과 한 글자도 다르지 않은지, 출처가 실제로 있는지, 위반 교체면 첫 문장이 보류 결론인지, 인정 경력의 출처 표기	규칙 기반 설명(출처 누락은 서버가 한 문장 덧붙임)
소명 글 (GraphRAG)	지식 그래프에서 만든 사실 목록 {F#} 중 무엇을 쓸지 고르고, 정해진 문법(주어·라벨·연결 말·맺음말)으로만 엮음. 숫자·충족 여부는 서버가 채움	없는 자리표시, 허용 목록 밖 연결 말, 팀원 아닌 ID, 사실과 라벨의 짝, 사람별 사실 누락. AI가 빠뜨린 불리한 사실은 서버가 마지막 문단에 덧붙임	정해진 틀의 글

소명 글은 처음에 자유 문장 + 금지어 목록으로 시작했으나 검토 때마다 뚫려, "AI는 사실 번호와 정해진 연결 말만 고르고 문장의 사실 부분은 서버가 채운다"로 바꿨다(E7b 94.7% → E7c 99.0%).

6.6 지식 그래프와 기술 이름 사전

- **기술 이름 사전**(개념 228개): "postgres"·"포스트그레스"·"PostgreSQL 14"처럼 갈린 이름과 버전 표기·괄호 병기를 한 이름으로 묶는다. 사전에 없는 이름은 합치지 않고 경고만 한다.
- **하위 기술 부분 인정**: 하위 기술 경력의 $0.7^{\text{깊이}}$를 상위 기술에 인정(Spring → Java)하고, 인정으로 정해진 경력은 출처·개월을 따로 남겨 화면·설명·소명 글에서 본인 경력과 구분한다. 사전 버전과 인정 비율은 계산 버전에 들어가, 바꾸면 이전 캐시·미리 계산이 쓰이지 않는다.

- **지식 그래프**(`core/kg`): 사업·요구 기술·팀원·협업·후보 비교·규칙 준수를 사실로 만들어 소명 글에 쓰고, 기술마다 실무 가능·숙련·비어 있는 사람·요구 사업을 모아 조직 기술 지도를 만든다. 저장 기술은 측정으로 골랐다(8월 Neo4j 우위 0/20 → 제거, 10월 RDF 표준 추론은 가중 부분 인정을 표현하지 못해 인메모리 그래프 유지).

7. 주요 설계 판단과 근거

판단	비교한 대안	근거(측정)	결정
서비스 솔버	CBC · SCIP · HiGHS	솔버당 112건 중 시간 안 완료 22 · 75 · 79(CBC는 품질 확인 불가). 규모 리허설에서 CBC는 세 규모 모두 최선 1% 안에 못 듦	HiGHS 고정
협업 보상 쌍 수	5,000쌍	200명 600초에도 빈자리 57석, 300명 해 없음 → 200쌍으로 모든 규모 빈자리 0	|C| 상위 200쌍
"익숙한 쌍" 기준	10년 중 6개월 · 5년 중 12개월 · 3년 중 12개월	300명 익숙한 쌍 11,733 · 1,352 · 395개. 10년 기준은 200명에서 빈자리 79석	최근 3년 중 12개월
오래 붙어 다닌 사이	쌍 감점 · 새 동료 수 하한 (문헌의 파트너 다양성)	하한은 SI 팀 크기에서 거의 걸리지 않고 200명 풀이를 망침. 감점은 익숙한 쌍을 200명 29 → 6, 300명 46 → 5로 흩고 품질 손실 −3.2% · −0.05%	쌍 감점 0.2 유지
투입 방식	투입률 나누기 · 동시 수행 인원 상한 · 감점 · 전일 투입	나누기는 잘 풀수록 쪼개기가 늘고, 상한·감점은 쪼개기를 못 막았다. 전일 투입은 1분 안 최선 증명(3.1)	전일 투입 + 사람 단위 허용(1.5)
AI의 역할	LLM 직접 배치	100명에서 규칙 위반 또는 솔버의 55%, 6~11분(3.3)	결정은 솔버, AI는 읽기·쓰기
평가 글 판정	규칙(항목만) · Jev · GLM-5.3-Flash · OpenAI	글로만 드러난 부정 12% · 94% · 100% · 100%, 돌려 말한 부정 — · 17% · 52% · 89%. 규칙이면 나쁘게 쓴 사이 22~34쌍이 협업 보상에 든다(3.4)	LLM(OpenAI) 판정
소명 글 방식	정해진 틀(템플릿) · GraphRAG	근거 오류는 둘 다 0, 읽기 쉬운 쪽으로 — 템플릿은 실패 시 대체 글로 유지	GraphRAG
숙련도 표현	경력 개월 비율 직접 사용	가상 데이터로는 우열을 가릴 수 없고 바꾸는 비용이 큼(같은 구간 안 차이는 안 보이는 한계를 기록)	대리 레벨 유지
추가 점수 항목	자리당 적합도 항 · 팩터 6종	측정 후 개선 근거 없음	채택 안 함(꺼진 선택지로 보존)

각 결정은 사용자 확인을 거쳤고, 측정 문서는 `rehearsal/results/`(rule-compare, partner-compare, llm-vs-solver …)·`docs/demo-notes.md`·`docs/skill-score-decision.md`에 있다. 하위 기술 인정 70%는 근거 데이터가 없는 가정값이다.

8. 사업적 가치와 확장

8.1 누가, 얼마나 자주 쓰는가

장면	쓰는 사람	빈도	이 시스템이 바꾸는 것
연초 계획	인력 운영 담당·사업부장	연 1~2회	조건을 모두 지키는 안 4개를 10분 안에 비교(300명 5.7분 · 연간 100명 9.0분), 최선과의 거리까지 표시
신규 사업 편성·긴급 투입	부서장·인력 운영 담당	신규 발주·이슈 때마다(수시)	"몇 명만 옮기면 되나"를 K=0~3 모두 최선 증명으로, 300명도 전체 약 1분(K 하나당 35초 안, 3.2)
교체 검토·보강	사업 책임자(PM)	수시	교체 전후 점수·위반과 근거 인용 설명을 몇 초 안에
인사팀 검토 자료	부서장 → 팀장·인사팀	확정마다 사업 수만큼	사업별 소명 자료: 잠긴 시스템 사실(다른 사업에서 옮겨 온 사람이면 원래 사업이 잃는 것·빈자리를 누가 채웠는지까지) + AI 초안을 부서장이 고친 본문 + 사실마다 해명·이의, 사람마다 판단·원래 사업 합의 → PDF

8.2 비용 구조(측정값)

- **솔버 라이선스 0원** — 오픈소스 HiGHS로 300명 안 A를 최선 대비 0.8% 안에서 푼다(111초에 증명, 대안 B~D 2.2~2.4%). 상용 솔버(Gurobi)는 연간 견적제이고, 무료판은 변수 2,000개 제한이라 100명 모델도 넘는다.

- **AI 호출 비용은 무시할 수준** — 300명 조직의 평가 글 4,012건을 처음 판정하는 데 약 $0.32(이후 저장값 재사용), 사업 60건 소명 글 전체 약 $0.05, 교체 설명 한 건 약 $0.0006.
- **서버 한 대** — 개발용 데스크톱 한 대에서 300명 계산·PDF까지 돌렸다. 사내 LLM으로 바꿀 때의 차이도 미리 쟀다(3.6).

8.3 효과 추정 모형

연간 절감 시간 = $\Sigma_{\text{장면}}$ (연간 횟수 × (수작업 시간 − 도구 사용 시간)). 아래는 300명 조직을 가정한 **예시 값**이다 — 도입 때 실제 업무 시간으로 바꿔 넣는다.

장면	연간 횟수	수작업(가정)	도구 사용(가정)	연간 절감
연초 계획(담당 3명)	2	72시간	8시간	128시간
신규 사업 편성·긴급 투입	24	6시간	1시간	120시간
인사팀 소명(사업 60건)	120	1시간	10분	100시간
합계				**약 350시간**

시간 절감보다 큰 효과는 결정의 질이다 — 같은 조건에서 빈자리·예산 초과 없는 배치(3.1), 나쁘게 평가한 사이에 협업 보상을 주지 않는 판정(3.4), 결정마다 남는 근거 문서. 실제 도입 때 이 효과를 실제 지표로 잰다(8.4).

8.4 차별점과 도입·확장 경로

- **차별점**: ① SI의 등급 정원·월 예산·휴직·협업 이력을 현업 규칙(전일 투입·사람 단위 동시 수행) 그대로 하나의 최적화로 풀고 증명 상한과 독립 검증으로 "얼마나 믿어도 되는지"까지 보인다. ② 가장 잦은 실무 결정인 "최소 인원 이동 재편성"을 최선 증명으로 푼다. ③ AI는 검사받는 부품이라 설명 글을 자동화하면서도 근거를 대조한다.
- **실제 도입 시 단계**(이 PoC의 범위 밖 — PoC는 가상 데이터로만 개발·검증했다): 섀도 운영(실제 편성과 나란히 3개월, 빈자리·예산 초과·재배치 횟수·고객 교체 요청·편성 소요 시간 기록) → 한 사업부 파일럿 → 전사·그룹사 확대. 그때 필요한 작업은 첫 데이터 적재(칸 이름 매핑·기술 이름 확인), 사내 LLM 연결, 계정·권한·감사·배포 같은 운영 기능이다.
- **결재 연결**(실제 도입 시): 이 서비스는 검토 자료(PDF·링크)를 만들어 **사내 전자결재**에 올리고 결과를 받아 반영한다. 역할은 나눈다 — 부서장(편성 담당)은 계산·명단 수정·검토 요청, 팀장·인사팀(검토자)은 같은 화면을 **읽기 전용**으로 보고 의견·승인·반려(사유 필수)만 한다(검토자가 명단을 바꿀 수 없다). 흐름은 작성 중 → 검토 요청 → 팀장 승인 → 인사팀 승인 → 확정이고, 실제 배치는 마지막 승인 때만 바뀐다. 검토 요청 시점의 자료는 서버 서명으로 고정하고 그 사이 데이터가 바뀌면 승인을 무효로 해 다시 요청한다. 원래 사업 측 동의(팀장·부사장·인사팀 경유 등)는 자료의 "원래 사업이 잃는 것"·"합의" 칸이 근거가 된다. 계정은 사내 SSO. 이 PoC의 확정은 검토 요청 없이 바로 반영하는 단순화다.
- **효과 입증 경로**(실제 도입 시): 승인된 과거 성과(고객 평가·교체 요청·후속 수주)로 모델 가정을 점검하는 도구(`core/evaluate/outcome_check.py`)와 계산 버전 관리가 이미 있어, 실제 성과가 쌓이면 계수를 보정하고 효과를 숫자로 보일 수 있다.
- **확장성**: 입력 계약·평가기·검증기·지식 그래프는 업종에 묶이지 않아, 사람을 프로젝트 단위로 배치하는 조직(컨설팅·엔지니어링 등)에 같은 구조로 쓸 수 있다.

9. 한계와 다음 단계

- **사업 효과는 실제 성과 자료로 확인하기 전이다**(`NOT_CALIBRATED`). 모든 수치는 가상 데이터와 설계한 점수표 안에서 잰 값이다 — 실제 도입 시 섀도 운영(8.4)으로 확인한다.
- 점수의 계수(협업 0.3·익숙한 사이 0.2·하위 기술 70%·경력 레벨 경계)는 설계값이다. 실제 성과로 보정할 수 있게 계산 버전과 평가기를 분리해 두었다.
- 평가 글은 생성기가 쓴 가상 글이다. 실제 글로 판정 정확도를 다시 잰다(E4 측정 도구 그대로).
- 운영 기능은 시제품 수준이다(관리자 계정 하나·단일 서버). 전일 투입은 가용률 0 또는 1을 전제로 한다(시간제는 실제 도입 시 다시 정한다).

부록. 재현 방법과 근거 파일

```
uv sync && scripts/run_poc.sh   # 실행 → http://127.0.0.1:8000 · 시험: uv run --group benchmark pytest -q (1,962 passed)
```